

Week 3 Problem Set

Classification, Regression Trees, Ensemble Methods & Regularisation

Release Date	June 18, 2026
Questions	8 (+ 1 Bonus)
Tools Required	Python, scikit-learn, XGBoost, Kaggle account
Submission	Jupyter Notebook (.ipynb) + written report (PDF)
Difficulty Key	[Easy] / [Medium] / [Hard]

How to Use This Problem Set

1. Each question targets a specific model or concept from the Week 3 study resource.
2. Each question uses a *different*, purpose-matched Kaggle dataset to maximise variety.
3. Download datasets from the Kaggle links provided. A free account is required.
4. Implement each model in a clearly labelled Jupyter Notebook section.
5. For every model: train on the training split, tune on validation, report final metrics on the held-out test set only.
6. **Random seed:** use `random_state=42` throughout for full reproducibility.

CONTENTS

1	Classification	2
1.1	Logistic Regression for Credit Default Prediction [Easy]	2
1.2	Decision Tree Classifier with Pruning for Heart Disease [Medium]	2
2	Regression	3
2.1	Regularised Regression for House Price Prediction [Medium]	3
2.2	Locally Weighted Regression on Non-Linear Data [Medium]	4
3	Ensemble Methods	5
3.1	Random Forest with Feature Importance for Fraud Detection [Hard]	5
3.2	XGBoost for Stock Return Classification [Hard]	6
4	Bias–Variance Analysis and Model Comparison	7
4.1	Empirical Bias–Variance Decomposition across Five Models [Medium]	7
5	Gradient Boosting in a Portfolio Context	8
5.1	Gradient Boosting with Uncertainty Estimation for Return Prediction [Hard]	8
	Bonus: Stacked Generalisation	9
	General Guidelines and Submission Instructions	10

1. CLASSIFICATION

1.1. Logistic Regression for Credit Default Prediction [Easy]

Background

Credit scoring is a canonical application of logistic regression in finance. You will build a binary classifier to predict whether a borrower will experience serious financial distress within two years. The dataset contains 150,000 loan records with ten financial features.

Tasks

1. Download the *Give Me Some Credit* dataset from Kaggle. Load `cs-training.csv` using pandas.
2. Perform the following pre-processing steps:
 - (a) Handle missing values in `MonthlyIncome` and `NumberOfDependents` using median imputation.
 - (b) Apply a stratified 70 / 15 / 15 train / validation / test split to account for class imbalance.
 - (c) Standardise all features using `StandardScaler` fitted *only* on the training set.
3. Train a `LogisticRegression` model using `solver='lbfgs'` and `max_iter=1000`. Evaluate on the test set and report the following metrics in a table:

Accuracy, Precision, Recall, F_1 -Score, AUC-ROC.

4. Plot the ROC curve and the confusion matrix side by side.
5. Vary the classification threshold $\tau \in \{0.1, 0.2, \dots, 0.9\}$. On a single figure, plot Precision and Recall as functions of τ . Mark the threshold that maximises F_1 .
6. Examine the trained coefficient vector $\hat{\mathbf{w}}$. Identify the five features with the largest $|\hat{w}_j|$ and interpret their financial meaning in 3–5 sentences.

Deliverables

- Cleaned dataset and pre-processing pipeline
- Trained logistic regression model
- Metrics table, ROC curve, and confusion matrix
- Threshold analysis plot
- Written interpretation of coefficient magnitudes in the credit-risk context

Hint. *Cross-entropy loss is convex in $(\mathbf{w}, b)$ for logistic regression, so gradient descent is guaranteed to converge to the global minimum. MSE applied to sigmoid outputs produces a non-convex landscape with many saddle points.*

1.2. Decision Tree Classifier with Pruning for Heart Disease [Medium]

Background

Decision trees are highly interpretable—a property of great value in healthcare and regulated financial settings. This question explores how depth and post-pruning affect

generalisation, and compares the two strategies numerically.

Tasks

1. Download the *Heart Disease UCI* dataset. Load `heart.csv`. Use `target` as the binary label.
2. Apply a stratified 70 / 15 / 15 split. Feature scaling is not required for decision trees.
3. Train an unpruned `DecisionTreeClassifier` (`max_depth=None`). Report training accuracy and test accuracy. Comment on overfitting.
4. Train decision trees with `max_depth` $\in \{1, 2, 3, 5, 7, 10, 15, \text{None}\}$. For each depth record the training accuracy and validation accuracy. Plot both curves on the same axes. Identify the depth that maximises validation accuracy.
5. Retrain the best depth-limited tree on train + validation combined. Report its test AUC-ROC.
6. Apply cost-complexity pruning:
 - (a) Use `cost_complexity_pruning_path()` to obtain the sequence of α values and corresponding impurities.
 - (b) For each α , train a tree and record its validation AUC-ROC.
 - (c) Select the α^* that maximises validation AUC-ROC. Retrain and evaluate on the test set.
7. Visualise the final pruned tree using `plot_tree` or `export_graphviz` (display at most 4 levels).
8. State which pruning strategy achieved higher test AUC-ROC and explain why in 2–3 sentences.

Deliverables

- Depth vs. accuracy learning curve plot
- Pruned tree visualisation
- Comparison table: Unpruned vs. Depth-limited vs. Cost-complexity pruned (AUC-ROC, F_1 , tree depth)
- Brief written analysis of results

Warning. *Pre-pruning can be overly aggressive. A split that appears unhelpful in isolation may enable a highly useful split downstream (the horizon effect). Cost-complexity pruning avoids this by growing the full tree first.*

2. REGRESSION

2.1. Regularised Regression for House Price Prediction [Medium]

Background

This Kaggle competition dataset contains 79 features describing residential properties. Many features are highly correlated (e.g., `GrLivArea` and `TotalBsmtSF`), making it an ideal testbed for Ridge, Lasso, and Elastic Net regularisation.

Tasks

1. Download the dataset. Load `train.csv`. Target variable: `SalePrice`.

2. Pre-processing:
 - (a) Apply a log-transform to `SalePrice`: $y \leftarrow \log(1 + y)$.
 - (b) Drop features with more than 40% missing values.
 - (c) Impute remaining missing values: median for numeric features, mode for categorical features.
 - (d) One-hot encode all categorical features.
 - (e) Standardise all numeric features using `StandardScaler` fitted on the training set.
3. Train the following three models using 5-fold cross-validation to select the regularisation strength λ :
 - (a) **Ridge** (`RidgeCV`): search $\lambda \in \{10^{-3}, \dots, 10^3\}$ on a log scale (50 values).
 - (b) **Lasso** (`LassoCV`): same λ grid.
 - (c) **Elastic Net** (`ElasticNetCV`): `l1_ratio` $\in \{0.1, 0.5, 0.7, 0.9, 0.95, 1.0\}$.
4. For each model, report: optimal λ^* , number of non-zero coefficients, RMSE on the test set (back-transformed via $\exp(\hat{y}) - 1$), and R^2 .
5. Plot the Lasso coefficient path: x -axis = $\log(\lambda)$, y -axis = coefficient value. Annotate the last five features to reach zero as λ increases.
6. In 3–4 sentences, explain geometrically why Lasso produces sparse solutions while Ridge does not, and discuss the implications for feature selection in noisy financial datasets.

Deliverables

- Comparison table: OLS vs. Ridge vs. Lasso vs. Elastic Net
- Lasso coefficient path plot
- Best λ^* and number of active features for each method
- Written discussion on sparsity and financial data

Hint. Geometrically, the ℓ_1 constraint region is a hyperdiamond whose corners lie on the coordinate axes. The quadratic loss contours are more likely to be tangent to these corners than to the smooth ℓ_2 sphere, setting some coefficients exactly to zero.

2.2. Locally Weighted Regression on Non-Linear Data [Medium]

Background

Locally Weighted Regression (LWR) fits a separate local linear model for each query point, weighting nearby training observations more heavily. California housing data exhibits spatial non-linearity that makes it an informative testbed for bandwidth selection.

Tasks

1. Download the dataset. Load `housing.csv`.
2. Construct a **one-dimensional sub-problem**: use `median_income` as the sole feature and `median_house_value` as the target. Sample 2,000 rows uniformly at random (`random_state=42`). Apply a 70 / 30 train / test split.
3. **Implement LWR from scratch using NumPy**. For each query point $\mathbf{x}_q$:
 - (a) Compute sample weights using a Gaussian kernel:

$$w_i = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_q\|^2}{2\tau^2}\right).$$

(b) Solve the weighted least-squares problem:

$$\hat{\boldsymbol{\theta}}_q = (\mathbf{X}^\top \mathbf{W}_q \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W}_q \mathbf{y},$$

where $\mathbf{W}_q = \text{diag}(w_1, \dots, w_n)$.

(c) Include a bias column (column of ones) in $\mathbf{X}$.

4. Experiment with bandwidths $\tau \in \{0.05, 0.1, 0.3, 1.0, 3.0\}$. For each τ :
 - Plot the LWR fit over the training data.
 - Report test RMSE.
5. Compare your best LWR model against ordinary least squares (OLS) on the same one-dimensional problem. Report test RMSE for both.
6. For which τ does LWR most closely approximate OLS? Provide a mathematical justification.
7. (*Extension*) Apply LWR to all numeric features simultaneously. Briefly describe any practical difficulties encountered (e.g., computational cost, curse of dimensionality).

Deliverables

- LWR implementation using NumPy only (no scikit-learn for this model)
- Five bandwidth comparison plots
- RMSE table: LWR (best τ) vs. OLS
- Written discussion on the role of τ and its limit as $\tau \rightarrow \infty$

3. ENSEMBLE METHODS

3.1. Random Forest with Feature Importance for Fraud Detection [Hard]

Background

Fraud detection involves severe class imbalance and high-dimensional, heterogeneous features. Random Forests are widely deployed in production fraud systems due to their robustness and the interpretable feature importance scores they provide. This question also contrasts two distinct importance measures.

Tasks

1. Download the dataset. Load `train_transaction.csv`. Target: `isFraud`. **Sample 50,000 rows** uniformly at random (`random_state=42`) for computational tractability.
2. Pre-processing:
 - (a) Drop columns with more than 50% missing values.
 - (b) Encode categorical columns as integer codes.
 - (c) Impute remaining NaN values with column medians.
 - (d) Apply a stratified 70 / 15 / 15 split.
3. Train a `RandomForestClassifier` with `n_estimators=200`, `oob_score=True`, and `class_weight='balanced'`. Record the OOB score.
4. Tune `max_features` via 5-fold stratified cross-validation over `{sqrt, log2, 0.3, 0.5}`. Report mean AUC-ROC for each value. Select the best setting and retrain.

5. Compute and plot:
 - (a) Top-20 features by **Mean Decrease in Impurity** (MDI).
 - (b) Top-20 features by **Permutation Importance** computed on the validation set.
 Do the two rankings agree? Discuss any discrepancies in 2–3 sentences, with reference to the known bias of MDI toward high-cardinality features.
6. Evaluate the best model on the test set. Report AUC-ROC, the Precision-Recall curve, and Average Precision (AP). Justify in one sentence why AUC-PR is more informative than AUC-ROC under class imbalance.
7. Plot OOB error as a function of `n_estimators` from 1 to 200. At what point does the curve plateau?

Deliverables

- Trained Random Forest with tuned `max_features`
- MDI vs. Permutation Importance comparison plot
- OOB error learning curve
- AUC-ROC and AUC-PR on the test set with written interpretation
- Discussion of class-imbalance handling (`class_weight='balanced'`)

3.2. XGBoost for Stock Return Classification [Hard]

Background

This question connects directly to the UAPOML goal: predicting whether a stock will produce a positive 21-day forward return. XGBoost dominates quantitative finance benchmarks due to its second-order optimisation and built-in regularisation. Critically, financial data demands time-series-aware validation to avoid look-ahead bias.

Tasks

1. Download the dataset. Load `all_stocks_5yr.csv`.
2. Feature engineering (computed per stock from daily OHLCV data):
 - (a) 5-day, 10-day, and 21-day rolling returns.
 - (b) 21-day rolling volatility (standard deviation of daily returns).
 - (c) Volume-price ratio: `volume/close`.
 - (d) RSI-14 (Relative Strength Index with a 14-day window).
 - (e) **Target**: binary label—1 if the 21-day forward return > 0 , else 0.
3. **Time-based split** (mandatory): train on all observations before 2017-01-01; test on 2017-01-01 onwards. Do *not* use random splitting.
4. Train an `XGBClassifier` with early stopping (`eval_metric='auc'`). Use the last 6 months of the training window as a held-out evaluation set for early stopping.
5. Perform a manual grid search over the following hyperparameters, evaluating by validation AUC-ROC:

Hyperparameter	Candidates
<code>max_depth</code>	{3, 5, 7}
<code>learning_rate</code>	{0.01, 0.05, 0.1}
<code>subsample</code>	{0.6, 0.8}

6. Report test AUC-ROC and AUC-PR. Compare to a naïve majority-class baseline.

7. Plot XGBoost feature importances (`importance_type='gain'`). Which feature contributes most? Provide an economic interpretation.
8. Answer in 4–6 sentences: why is random k -fold cross-validation invalid for financial time series? Describe precisely the mechanism by which look-ahead bias inflates performance estimates.

Deliverables

- Feature engineering pipeline
- Time-based split implementation
- Hyperparameter grid search results table
- XGBoost model with best hyperparameters
- Feature importance plot
- Test AUC-ROC and AUC-PR vs. baseline
- Written answer on look-ahead bias (minimum 5 sentences)

Warning. *Never use `sklearn`'s `KFold` or `train_test_split` (without temporal ordering) on financial data. Doing so leaks future information into the training set and produces grossly optimistic performance estimates.*

4. BIAS–VARIANCE ANALYSIS AND MODEL COMPARISON

4.1. Empirical Bias–Variance Decomposition across Five Models [Medium]

Background

The bias–variance tradeoff is most instructively demonstrated by *measuring* how predictions vary across different training realisations. This question uses bootstrap resampling to empirically estimate Bias^2 and Var for five models that span the complexity spectrum.

Tasks

1. Download the dataset. Load `diabetes.csv`. Target: `Outcome`.
2. Apply a fixed 70 / 30 train / test split (`random_state=42`). The test set is **fixed** for all evaluations.
3. Empirical estimation procedure ($B = 50$ bootstrap iterations):
 - (a) Draw B bootstrap samples $\mathcal{D}_1^*, \dots, \mathcal{D}_B^*$ from the training set (with replacement).
 - (b) Train the model on each $\mathcal{D}_b^*$ and collect predictions $\hat{y}_i^{(b)}$ on the *full* test set.
 - (c) Estimate:

$$\widehat{\text{Bias}}^2 = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (\bar{\hat{y}}_i - y_i)^2, \quad \widehat{\text{Var}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \frac{1}{B} \sum_{b=1}^B \left(\hat{y}_i^{(b)} - \bar{\hat{y}}_i \right)^2,$$

where $\bar{\hat{y}}_i = \frac{1}{B} \sum_b \hat{y}_i^{(b)}$.

4. Apply this procedure to all five models:

Model	Configuration	Expected Regime
Logistic Regression	default	High Bias, Low Variance
Decision Stump	<code>max_depth=1</code>	High Bias, Low Variance
Unpruned Tree	<code>max_depth=None</code>	Low Bias, High Variance
Random Forest	<code>n_estimators=100</code>	Low Bias, Low Variance
XGBoost	default hyperparameters	Low Bias, Moderate Variance

5. Compile a summary table with columns: **Model** | $\widehat{\text{Bias}}^2$ | $\widehat{\text{Var}}$ | $\widehat{\text{Bias}}^2 + \widehat{\text{Var}}$ | **Test AUC-ROC**.
6. Produce a grouped bar chart with $\widehat{\text{Bias}}^2$ and $\widehat{\text{Var}}$ stacked for each model.
7. In 4–6 sentences, explain why bagging (Random Forest) reduces variance but does not substantially reduce bias, while boosting (XGBoost) reduces both bias and variance. Reference the formal variance formula $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ in your answer.

Deliverables

- Bootstrap estimation code
- Summary table for all five models
- Stacked bar chart ($\widehat{\text{Bias}}^2 + \widehat{\text{Var}}$)
- Written explanation of bagging vs. boosting effects on the bias–variance tradeoff

5. GRADIENT BOOSTING IN A PORTFOLIO CONTEXT

5.1. Gradient Boosting with Uncertainty Estimation for Return Prediction [Hard]

Background

This capstone question integrates gradient boosting, feature selection, time-series cross-validation, and—most relevantly to the UAPOML theme—*prediction uncertainty estimation* via ensemble disagreement. You will predict 21-day forward returns for NIFTY-50 constituents and use ensemble disagreement as a proxy for model uncertainty.

Tasks

1. Download the NIFTY-50 dataset. Target: binary label—1 if 21-day forward return > 0 , else 0 (shift by 21 trading days).
2. Feature engineering (minimum eight features per stock per date):
 - (a) Rolling returns: 5-day, 21-day, and 63-day.
 - (b) Rolling volatility: 21-day and 63-day standard deviations of daily returns.
 - (c) Price relative to 52-week high: `close/max252d(close)`.
 - (d) Price relative to 52-week low: `close/min252d(close)`.
 - (e) Volume z-score using a 21-day rolling mean and standard deviation.
3. Use an **expanding-window time-series cross-validation** (minimum 3 folds) to tune a `GradientBoostingClassifier`:

Hyperparameter	Candidates
n_estimators	{100, 200, 500}
max_depth	{2, 3, 5}
learning_rate	{0.01, 0.05, 0.1}

4. Feature selection: use permutation importance from the tuned model to identify the top five features. Retrain using only those five features. Compare test AUC-ROC of the full model against the five-feature model.
5. **Uncertainty estimation via ensemble disagreement:**
 - (a) Train $M = 20$ gradient boosting models on distinct bootstrap samples of the training set.
 - (b) For each test observation, compute the mean predicted probability $\bar{p}_i$ and standard deviation σ_i across the 20 models.
 - (c) Produce a scatter plot of $\bar{p}_i$ (x-axis) against σ_i (y-axis). Identify and annotate the ten observations with highest σ_i .
6. **Simple long-short strategy** (illustrative only): rank test-period observations by $\bar{p}_i$. Go long the top decile and short the bottom decile. Plot cumulative returns of this strategy against a buy-and-hold NIFTY-50 benchmark over the same period.

Disclaimer

This exercise is illustrative and does not account for transaction costs, slippage, or market impact. It should not be interpreted as a trading recommendation.

7. Reflection (5–8 sentences): What are the key risks of deploying this model in a live trading environment? Your answer must address: (i) overfitting to non-stationary financial data, (ii) look-ahead bias, (iii) transaction costs, and (iv) the relationship between σ_i (model uncertainty) and position sizing in an uncertainty-aware portfolio.

Deliverables

- Feature engineering pipeline
- Expanding-window CV results table
- Full model vs. five-feature model AUC-ROC comparison
- Uncertainty scatter plot ($\bar{p}_i$ vs. σ_i)
- Cumulative return chart vs. NIFTY-50 benchmark
- Reflection essay on real-world limitations (minimum 5 sentences)

BONUS: STACKED GENERALISATION FOR TITANIC SURVIVAL PREDICTION

Background

Stacking learns a meta-learner that optimally combines the predictions of diverse base models. This question asks you to implement the procedure from scratch (or using `sklearn`'s `StackingClassifier`) and compare the ensemble against each individual base learner.

Tasks

1. Download the *Titanic* dataset. Perform standard feature engineering (e.g., `Sex`, `Pclass`, `Age` with imputation, `Fare`, `Embarked`). Target: `Survived`.
2. **Level-0 base learners** (train using 5-fold out-of-fold predictions):
 - (a) Logistic Regression
 - (b) Decision Tree (`max_depth=5`)
 - (c) Random Forest (`n_estimators=100`)
 - (d) XGBoost (`n_estimators=100`)
 - (e) k -Nearest Neighbours (`k=5`)
3. **Level-1 meta-learner**: train a Logistic Regression on the $5 \times n_{\text{train}}$ meta-feature matrix of out-of-fold predictions. Evaluate by predicting the test set using base learner test predictions as meta-features.
4. Comparison table: each base learner individually **vs.** the stacked ensemble. Report AUC-ROC and Accuracy.
5. Plot the meta-learner coefficients. Which base learner receives the highest weight? Discuss whether this aligns with the individual test AUC-ROC rankings.
6. Submit your predictions to the Kaggle public leaderboard and report your score.
7. In 5–8 sentences, explain why out-of-fold predictions *must* be used to construct the meta-features. Describe precisely what happens to meta-learner training if in-sample predictions from overfit base learners are used instead.

Deliverables

- Stacking implementation (from scratch or `StackingClassifier`)
- Base learner vs. ensemble comparison table
- Meta-learner coefficient plot
- Kaggle public leaderboard score
- Written discussion on data leakage in stacking (minimum 5 sentences)

GENERAL GUIDELINES AND SUBMISSION INSTRUCTIONS**Code Quality**

- All code must be contained in a single Jupyter Notebook (`.ipynb`). Use clearly labelled Markdown cells for each question.
- Include inline comments explaining non-trivial design choices.
- The notebook must run end-to-end without errors (*Kernel → Restart & Run All*).
- Use `random_state=42` everywhere for reproducibility.

Report Structure

- Begin each question with a 2–3 sentence summary of what you are doing and why.
- End each question with a written interpretation of the results.
- All figures must have labelled axes, a title, and a legend where applicable.
- All tables must have column headers and clearly identified row labels.

Finance Context

- For Questions 6, 8, and the Bonus: explicitly connect your findings to the UAPOML theme of uncertainty-aware allocation. How does prediction uncertainty (σ_i) inform position sizing?
- When reporting results on financial datasets always state: (i) the evaluation period, (ii) the train/test temporal boundary, and (iii) any potential survivorship bias present in the data.

Academic Integrity

- You may use `scikit-learn`, `xgboost`, `lightgbm`, and standard Python data science libraries.
- The LWR implementation in Question 4 must be written from scratch using NumPy only.
- All written explanations must be in your own words. Uncited reproduction of external sources is not permitted.

Grading Rubric

Component	Criteria	Weight
Code Correctness	Models run, outputs are sensible, metrics computed correctly	30%
Written Analysis	Clarity, depth, and connection to finance context	25%
Visualisations	Appropriate plot types, labelled axes, readable and informative	15%
Methodology	Correct train/val/test protocol with no data leakage	20%
Reproducibility	Notebook runs end-to-end with fixed random seeds	10%
Bonus	Stacking correctly implemented; Kaggle leaderboard score reported	+15%

Good luck. In quantitative finance, rigorous validation is not optional.

UAPOML Study Resource — Week 3 Problem Set — June 2026